

ANÁLISIS DEL RETO

John Acosta, 202212004, ja.acostar1@uniandes.edu.co

Andres Zamora, 202516126, as.zamorar1@uniandes.edu.co

La repartición del trabajo fue: John Acosta (Requerimiento 2), Andrés Zamora (Carga de datos y requerimiento 1), y requerimiento 4,5 y 6 en conjunto. Debido a que en el documento del reto no se cuenta la carga de datos como requerimiento, no incluimos su análisis.

Requerimiento <<n>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	Parámetros necesarios para resolver el requerimiento.
Salidas	Respuesta esperada del algoritmo.
Implementado (Sí/No)	Si se implementó y quien lo hizo.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1	$O(\dots)$
Paso 2	$O(\dots)$
Paso	$O(\dots)$
TOTAL	$O(\dots)$

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

Plantilla para el documentar y analizar cada uno de los requerimientos.

Requerimiento 1

Descripción

Hacemos un for para filtrar los viajes, los viajes que cumplen con el filtro se guardan en un array list. Si la muestra el doble de la muestra dada es mayor que el tamaño total de nuestro arreglo, entonces cargamos todos los datos, en caso contrario solo cargamos los datos de la cantidad de la muestra. Para ambos casos cargamos los datos se cargan con un for que recorre la lista y guarda los datos en un diccionario que luego será el retorno.

Entrada	Datos, fecha inicial, hora final y la muestra.
Salidas	Un diccionario con toda la información para imprimir, esto incluye: tiempo, tamaño e información de los viajes seleccionados con respecto a la muestra. La información de los viajes es la fecha de recogida, sus coordenadas, la fecha de dropoff, sus coordenadas, la distancia del viaje y el costo total.
Implementado (Sí/No)	Sí, lo hizo Andrés Zamora.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Tomamos el tiempo, convertimos las fechas a datetime y hacemos una sublista (que realmente es una lista vacía), en general todo ocupa $O(1)$	$O(1)$
Hacemos un for que cumple con el filtro, tiene un if que tiene una complejidad de $O(1)$.	$O(N)$
Definimos un sort criteria	$O(1)$
Ordenamos con quick sort	$O(N * \log N)$
Creamos listas de apoyo que nos ayudaran despues y hacemos comparaciones de tamaño con if. Cada operación es $O(1)$	$O(1)$
Si la lista es mayor que 2 veces la muestra, entonces creamos 2 sublistas con la información de la lista hasta la muestra y otra desde la muestra hasta el final, cada una es $O(N)$.	$O(N)$

Hacemos for para arreglar la información que nos piden, son 2 for pero están separados, cada for es $O(N)$	$O(N)$
Cada for tiene un get, este get siempre va a ser $O(1)$ porque estamos usando array list.	$O(1)$
En caso de tener que mostrar todos los elementos, hacemos un único for para arreglar toda la información que necesitamos para el retorno.	$O(N)$
Creamos un diccionario y retornamos la información.	$O(1)$
TOTAL	$O(...)$

Análisis

Este requerimiento tiene una complejidad de $n * \log n$ debido al quick sort, el quick sort parte más o menos a la mitad la lista y luego recorre n veces, lo que nos da $n * \log n$. Consideramos que podría ser optimizado creando una función particularmente para hacer for y cargar los datos de forma más limpia.

Requerimiento 2

Descripción

Primero se recorren todos los viajes almacenados en el catálogo de taxis para filtrar aquellos cuya latitud de recogida se encuentre dentro del rango dado por el usuario (lat_min, lat_max). Una vez obtenidos los viajes válidos, se ordenan por latitud de recogida de mayor a menor, y en caso de empate, por longitud también de mayor a menor. Finalmente, se seleccionan los N primeros y los N últimos trayectos del conjunto ordenado (o todos si son menos de $2N$) y se formatea la información relevante para su visualización.

Entrada	Catalog, Coordenada inicial de latitud (lat_min), Coordenada final de latitud (lat_max), Tamaño de la muestra (sample_size).
Salidas	Retorna un diccionario llamado retorno que contiene tiempo, total de trayectos, N primeros y N últimos trayectos (o todos si hay menos de $2N$), Fecha y hora de recogida, Coordenadas de recogida , Fecha y hora de terminación, Coordenadas de terminación, Distancia recorrida, Costo total pagado.
Implementado (Sí/No)	Sí se implementó, lo hizo John Acosta

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se toma el tiempo inicial con <code>get_time()</code>	$O(1)$
Recorrer toda la lista de viajes (for sobre trips) y filtrar aquellos cuya latitud esté dentro del rango. Cada comparación y acceso al campo es	$O(N)$
Obtener el tamaño de la lista filtrada (<code>lt.size(filtered)</code>).	$O(1)$
Ordenar la lista de viajes filtrados mediante <code>merge_sort</code> . El ordenamiento tiene una complejidad promedio de $O(M \log M)$, donde M es el número de viajes filtrados.	$O(M \log M)$
Crear sublistas de tamaño N con <code>lt.sub_list</code> . Cada una recorre internamente los elementos seleccionados.	$O(N)$
Formatear los datos de los viajes seleccionados (ya sean todos, o los primeros y últimos). Se hace con comprensión de listas o bucles simples.	$O(N)$
Se crea el diccionario retorno con toda la información necesaria.	$O(1)$
TOTAL	$O(M \log M)$

Análisis

Escala de forma lineal por lo que no afecta demasiado el tamaño del csv que se quiera analizar, pero podría mejorarse en memoria simplificando los for y dejando de usar la lista met.

Requerimiento 3

Descripción

identificar los trayectos en un rango de la distancia recorrida de mayor a menor. Para trayectos con igual distancia el ordenamiento se hace de mayor a menor costo total pagado

Entrada	Catalog, Valor inicial, valor final y tamaño de la muestra.
Salidas	Retorna un diccionario llamado con el tiempo de ejecución en ms, numero de trayectos que cumplen ese filtro, tiempo promedio de los trayectos, costo total promedio, distancia promedio, costo promedio en peajes, numero y cantidad de pasajeros mas frecuentes en los trayectos del rango, cantidad de propina promedia, fecha de finalización del trayecto.
Implementado (Sí/No)	NO se implementó, A causa de que no hay estudiante 3

Requerimiento 4

Descripción

El requerimiento consiste en identificar los trayectos de taxi que finalizaron en una fecha específica, considerando además un tiempo de referencia que permite filtrar los viajes según si terminaron antes o después de dicha hora dentro de la fecha indicada. Para lograrlo, primero se organiza toda la información de los trayectos dentro de una tabla de hash, donde cada llave representa una fecha de terminación en formato "YYYY-MM-DD", y el valor asociado es una lista con todos los trayectos terminados en esa fecha.

Entrada	Fecha de terminación de los trayectos, en formato AAAA-MM-DD, Indicador de tiempo: ANTES o DESPUÉS, según el tipo de comparación que se desea realizar respecto al tiempo de referencia. Tiempo de referencia, en formato HH:MM:SS y Tamaño de la muestra N,
Salidas	El resultado es un diccionario con toda la información necesaria para imprimir y analizar los datos. Incluye el tiempo total de ejecución del requerimiento en milisegundos, el número total de trayectos que cumplen las condiciones de fecha y hora, y dos listas con los trayectos seleccionados: los primeros N y los últimos N (o todos, si hay menos de 2N). Cada trayecto muestra la fecha y hora de recogida, las coordenadas de recogida y terminación, la fecha y hora de terminación, la distancia recorrida en millas y el costo total del viaje.
Implementado (Sí/No)	Sí se implementó, lo hizo John Acosta y Andres Zamora

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se toma el tiempo inicial con get_time()	$O(1)$
Recorrer todos los trayectos del catálogo y agregar cada uno en la posición correspondiente dentro de la tabla de hash según su fecha de terminación. Dado que se recorre toda la lista de viajes una vez, este paso tiene complejidad lineal.	$O(N)$,

Buscar la lista asociada a la fecha ingresada por el usuario dentro de la tabla hash. La búsqueda tiene un tiempo promedio constante.	$O(1)$
Filtrar los trayectos según el parámetro de tiempo ("ANTES" o "DESPUÉS") comparando la hora de terminación con la hora de referencia. Cada comparación se realiza una vez por trayecto dentro de la lista seleccionada.	$O(M)$
Ordenar los trayectos filtrados de más reciente a más antiguo, utilizando un algoritmo de ordenamiento eficiente (como merge sort), cuya complejidad promedio es $O(M \log M)$.	$O(M \log M)$
Seleccionar las sublistas con los primeros y últimos N trayectos (o todos si son menos de $2N$). Estas operaciones sobre listas son proporcionales al tamaño de las sublistas.	$O(N)$
Formatear la información de los trayectos seleccionados para generar la salida final. Se hace mediante recorridos simples sobre las sublistas.	$O(N)$
Se construye el diccionario de salida con los resultados	$O(1)$
TOTAL	<i>El paso que domina la complejidad es el ordenamiento de los trayectos filtrados, por lo que el requerimiento tiene una complejidad total de $O(M \log M)$, donde $M \leq N$ representa la cantidad de trayectos que cumplen el filtro de fecha y hora.</i>

Análisis

Este requerimiento es eficiente gracias al uso de una tabla de hash, que permite acceder de forma directa a los trayectos correspondientes a una fecha específica sin tener que recorrer todos los datos. El filtrado por hora y el ordenamiento posterior aseguran que la salida esté correctamente estructurada y sea fácil de interpretar. Aunque el algoritmo tiene un paso de ordenamiento que domina el tiempo total, la estrategia de almacenamiento en hash optimiza el proceso de búsqueda inicial. En escenarios con grandes volúmenes de datos, esta estructura reduce significativamente el tiempo de acceso a las listas de trayectos relevantes.

Requerimiento 5

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Primero guardamos todos los viajes en una tabla de hash, en la cual cada llave es una fecha en el formato año, mes, día y hora, y el valor asociado a cada llave es una lista con los viajes en esas horas. Después buscamos la fecha que necesita el usuario, tomamos la lista asociada a la llave. Ordenamos la lista de forma ascendente y después hacemos for para arreglar la información que debemos retornar.

Entrada	Datos, fecha y muestra.
Salidas	Un diccionario con toda la información para imprimir, esto incluye: tiempo, tamaño e información de los viajes seleccionados con respecto a la muestra. La información de los viajes es la fecha de recogida, sus coordenadas, la fecha de dropoff, sus coordenadas, la distancia del viaje y el costo total.
Implementado (Sí/No)	Sí, lo hizo Andrés Zamora y John Acosta.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Tomamos el tiempo, convertimos la fecha a datetime y copiamos el catalog de taxis para facilitar el manejo de datos, y creamos la tabla de hash, cada operación es $O(1)$	$O(1)$
Guardamos los datos en la tabla de hash, usando las fechas como llaves y los valores son listas con cada viaje. Al usar un for es $O(n)$, el resto de las operaciones adentro del for son $O(1)$	$O(N)$
Buscamos cual lista es la que necesita el usuario usando la fecha como llave, después ordenamos la lista con merge sort.	$O(n * \log n)$
Si $2 * \text{muestra}$ es mayor que nuestra lista, creamos 2 sublistas.	$O(N)$
Seguido a esto, hacemos 2 for independientes para arreglar los datos de los últimos y de los primeros. Cada for es $O(N)$	$O(N)$
En caso contrario solo hacemos 1 for para todos los datos.	$O(N)$
Creamos un diccionario y retornamos la información.	$O(1)$

TOTAL	$O(...)$
--------------	----------------------------

Análisis

La complejidad de este requerimiento es $O(N * \log N)$ por el ordenamiento merge, esto debido a que merge parte la lista en sublistas hasta que solo haya 1 elemento, ahí compara y ordena, siempre parte en la mitad si la lista tiene un tamaño par, por lo que la complejidad se vuelve $n * \log n$. Consideramos que podría ser optimizado creando una función particularmente para hacer for y cargar los datos de forma más limpia. Además, podría hacerse una mejor forma si no limitasen el uso de la tabla de hash.

Requerimiento 6

Descripción

El objetivo es identificar los trayectos cuyo punto de recogida corresponde a un barrio de Nueva York determinado y cuya hora de recogida se encuentra dentro de un rango horario especificado por el usuario. Para asignar cada viaje a un barrio se calcula la distancia entre la coordenada de recogida y los centroides de los barrios, seleccionando el barrio con la distancia más pequeña mediante la fórmula de Haversine (distancia en millas). Posteriormente, se filtran los viajes asignados al barrio solicitado por la hora de recogida considerando únicamente la hora (HH) y se ordenan cronológicamente de más antiguo a más reciente. Si el número total de trayectos filtrados es menor o igual a $2N$, se muestran todos; en caso contrario se muestran los N primeros y los N últimos.

Entrada	Catalog, <i>barrio_inicio</i> , <i>Hora_inicio</i> , <i>Hora_fin</i> , tamaño N
Salidas	Retorna un diccionario con, tiempo de ejecución en ms, total de trayectos en el rango de fechas, promedio de distancia recorrida, Si total $> 2N$: dos listas con los N primeros y los N últimos trayectos (ordenadas de antiguo a reciente). Si total $\leq 2N$: una sola lista con todos los trayectos hallados. Cada trayecto devuelto incluye: fecha y hora de recogida, coordenadas de recogida, fecha y hora de finalización, coordenadas de finalización, distancia recorrida (millas) y costo total.
Implementado (Sí/No)	Sí se implementó, lo hizo John Acosta y Andres Zamora

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se toma el tiempo inicial con <code>get_time()</code>	$O(1)$
Normalización de parámetros de hora: Convertir las entradas de hora (<code>start_hour</code> , <code>end_hour</code>) a enteros en formato 0–23 y validar que estén dentro del rango permitido; en caso de formato inválido devolver un mensaje de error.	$O(1)$,
Se define la función <code>haversine()</code> para calcular la distancia entre coordenadas y la función <code>nearest_neighborhood()</code> para buscar el barrio más cercano a un punto.	$O(1)$
Se recorre la lista de todos los viajes del catálogo (N elementos).	$O(N)$
Para cada viaje, se busca el barrio de recogida más cercano comparando con todos los centroides de barrios (M).	$O(N*M)$
Se insertan los viajes en la tabla hash correspondiente al barrio. Cada inserción o actualización tiene costo promedio constante.	$O(1)$
Se recupera la lista de viajes del barrio indicado por el usuario desde la tabla hash.	$O(1)$
Se filtran los viajes del barrio por la hora de recogida, considerando solo la parte de la hora (HH).	$O(M)$
Se ordenan los viajes filtrados por fecha y hora de recogida (de más antiguo a más reciente) usando Merge Sort.	$O(M \log M)$
Se construye el diccionario de salida con los resultados	$O(1)$
Se seleccionan los primeros N y últimos N viajes (o todos si $M' \leq 2N$) y se formatean los datos para el resultado.	$O(N)$
Se construye el diccionario de salida con los resultados a mostrar.	$O(1)$
TOTAL	$O(N*M)$

Análisis

En este requerimiento, el paso más costoso corresponde a la búsqueda del barrio más cercano para cada trayecto, ya que por cada viaje del catálogo (N) se recorren todos los centroides de los barrios (M) para determinar cuál se encuentra a menor distancia, utilizando la fórmula de Haversine. Este proceso genera un costo total de $O(N \cdot M)$, que domina la complejidad global del algoritmo. Las operaciones posteriores, como la inserción de los viajes en la tabla hash, la recuperación de los trayectos pertenecientes al barrio seleccionado y el filtrado por hora de recogida, tienen costos significativamente menores ($O(1)$ o $O(M)$). El ordenamiento cronológico de los trayectos filtrados, aunque tiene una complejidad $O(M \log M)$, se aplica sobre un subconjunto mucho más pequeño de datos, por lo que no afecta de manera considerable el tiempo total de ejecución. Asimismo, el uso de una tabla hash permite acceder de forma eficiente a los viajes agrupados por barrio, reduciendo la necesidad de recorrer toda la lista de trayectos en búsquedas posteriores. Sin embargo, el cálculo repetido de distancias hacia todos los barrios constituye el principal cuello de botella del requerimiento. Como posible optimización, se podría precalcular el barrio asociado a cada punto de recogida durante la carga inicial del catálogo o implementar estructuras espaciales como KD-Trees o R-Trees.